index

A

abstraction 112

Accept-Language header 77

access tokens

propagating 265–270

refreshing 390

access_token attribute 257

action element 290

ActionEnum 289

ActiveCycle setting 200

ActivePermissions setting 200

actuator/gateway/refresh endpoint 221

actuator/gateway/routes endpoint 217, 221

add() method 80

admin processes 45–46

Amazon

creating EC2 instances with ELK 353–356

creating PostgreSQL database using Amazon RDS 348–350

creating Redis cluster in 351–352

deploying ELK stack in EC2 instance 356–357

Amazon’s EKS Container Service 366

AMI (Amazon Machine Image) 353

AOP (Aspect-Oriented Programming) 186

Apache Kafka, configuring in Docker 285

API Gateway server 213

API Gateways 382

ApiGatewayServerApplication class 215

APM (Application Performance Management) 327

Application class 11

application metrics pattern 27–28

ApplicationContext 119

architect's role 56–62

decomposing business problem 56–59

defining service interfaces 61–62

establishing service granularity 59–61

Aspect-Oriented Programming (AOP) 186

AtomicRateLimiter 200

AUTH_TOKEN variable 271

authentication 25

authentication server 243

authorization 25

authorization grant types 386–388

Authorization header 266, 268

authorization server 243

B

backing services 42

baseUrl property 328

@Bean tag 53

@Beans annotation 76

best practices

microservices architecture

API Gateways 382

CI/CD 380–381

externalized configuration 380

logging 382

monitoring 381

Richardson Maturity Model 378–380

Spring HATEOAS 380

twelve-factor app 37–46

admin processes 45–46

backing services 42

build, release, run 42–43

codebase 40

concurrency 44

config 41

dependencies 40

dev/prod parity 44–45

disposability 44

logs 45

port binding 43

processes 43

binder 284

bootBuildImage task 105

bootstrap class

Spring Boot 52–53

Spring Cloud Configuration Server 120

bridge type 95

build and deployment patterns 28

build, release, and run stages 42–43

build/deployment pipeline 376

architecture of 342–346

creating 366–374

creating Kubernetes pipeline scripts 374

enabling services to build in Jenkins 367–372

setting up GitHub 366–367

understanding and generating pipeline script 372–374

in action 364–366

buildFallbackLicenseList() method 194

Buildpacks 105

@Bulkhead annotation 185, 198, 205

bulkhead pattern 24

implementing 195–198

overview 180–181

business problem, decomposing 56–59

C

CaaS (Container as a Service) 16

cacheOrganizationObject() method 303

CallNotPermittedException 188

CD (continuous delivery) 343, 380

checkRedisCache() method 303

CI (continuous integration) 342, 380

CI/CD (continuous integration and continuous delivery) 344, 380–381

circuit breaker pattern 187–193

adding to organization service 191–192

customizing 192–193

overview 180

@CircuitBreaker annotation 185, 189–191, 194, 196, 205

classpath attribute 122

client credential grant types 385

client resiliency patterns 23–207

bulkhead pattern 180–181, 195–198

circuit breaker pattern 180, 187–193

fallback processing pattern 180, 193–194

importance of 181–184

load balancing pattern 179–180

rate limiter pattern 200–202

Resilience4j

implementing 185

setting up licensing service to use 186–187

ThreadLocal and 202–207

retry pattern 198–200

client-side load balancing 24, 154

clientType parameter 169

cloud computing 11–19

building microservices with Spring Boot 11–14

cloud-native microservice app best practices 37–46

admin processes 45–46

backing services 42

build, release, run 42–43

codebase 40

concurrency 44

config 41

dependencies 40

dev/prod parity 44–45

disposability 44

logs 45

port binding 43

processes 43

defined 16–17

reasons for deploying microservices to cloud 18–19

service discovery 152–157

architecture 153–156

using Spring and Eureka 156–157

setting up core infrastructure in cloud 346–352

creating PostgreSQL database using Amazon RDS 348–350

creating Redis cluster in Amazon 351–352

cloud-native application 37

cloud-ready application 37

ClusterIP service 362

codebase 40

@Column attribute 134

@Component annotation 53

concurrency 44

config.getProperty() class 137

configuration drift 28

@Configuration tag 53

@ConfigurationProperties 137–138

configure() method 262

consistency 63

Consul 33

Container as a Service (CaaS) 16

containers

container sprawl 63

virtual machines vs. 92–94

continuous delivery (CD) 343, 380

continuous integration and continuous delivery (CI/CD) 344, 380–381

@Controller annotation 65

Controller class 66

Conway's law 7

core development pattern 21–22

coreThreadPoolSize 197

correlation ID

adding to HTTP response with Spring Cloud Gateway 325–326

building post-filter receiving 237–238

Spring Cloud Sleuth and 309–311

adding to licensing and organization 309–310

anatomy of trace 310–311

UserContext 234–235

UserContextFilter 233–234

correlationId element 290

createLicense() method 71, 77

credential management and propagation 25

cross-cutting concerns 222

CRUD (Create, Read, Update, Delete) 61, 134, 301

CrudRepository class 135

CustomChannels interface 304–305

D

data transactions 63

decomposing 4

/decrypt endpoint 146

@DeleteMapping annotation 71

dependencies 40

setting up licensing service dependencies 128–129

setting up Spring Cloud Sleuth and Zipkin dependencies 328

twelve-factor app best practices 40

dependency injection framework 7

dependency:tree goal 51

deploying microservices 376

build/deployment pipeline 376

architecture of 342–346

creating 366–374

in action 364–366

deploying O-stock 353–364

creating EC2 instances with ELK 353–356

creating EKS clusters 357–364

deploying ELK stack in EC2 instance 356–357

setting up O-stock core infrastructure in cloud 346–352

creating PostgreSQL database using Amazon RDS 348–350

creating Redis cluster in Amazon 351–352

development/production parity 44–45

DevOps (development operations) 38

DevOps engineer's role 81–89

communicating microservice health 86–89

service assembly 83–84

service bootstrapping 84–85

service registration and discovery 85–86

DiscoveryClient class 172

disposability 44

distributed caching 297–306

distributed tracing 339, 382

Spring Cloud Sleuth

correlation ID and 309–311

log aggregation and 311–326

Zipkin 327–338

adding custom spans 336–338

capturing messaging traces 334–335

configuring server 328–329

configuring services to point to 328

setting tracing levels 329–330

setting up dependencies 328

tracing transactions 330–333

visualizing more complex transactions 333

Docker 109

adding Keycloak to 245

configuring Apache Kafka and Redis in 285

containers vs. virtual machines 92–94

defined 94–96

defining and running ELK stack applications in 318–320

Docker Compose 97–98

integrating with microservices 99–108

creating Docker Images 99–104

launching services with Docker Compose 107–108

Docker client 95

docker commit command 108

Docker Compose 97–98, 107–108

Docker containers 95

Docker daemon 95

Docker Images

creating 99–104

basic Dockerfiles 100

multistage build Dockerfiles 101–104

Buildpacks 105

Docker images 95

docker images command 104, 359

Docker networks 95

Docker registry 95

docker run command 95, 104, 141

docker tag command 360

Docker volumes 95

docker-compose command 98, 124, 162, 320, 367

docker-compose config command 97

docker-compose up command 97, 108, 124

docker-machine command 367

basic 100

multistage build 101–104

doFilter() method 205

E

EC2 (Elastic Compute Cloud)

creating instances with ELK 353–356

deploying ELK stack in instance 356–357

ECR (Elastic Container Registry) 358, 365

ECS (Elastic Container Service) 16

EDA (event-driven architecture) 276–306

messaging 279–281

downsides of 281–282

durability 280

flexibility 281

loose coupling 280

scalability 280–281

Spring Cloud Stream 282–297

configuring Apache Kafka and Redis in Docker 285

defining custom channels 304–306

distributed caching 297–306

using Redis to cache lookups 298–304

writing message consumer in licensing service 293–295

writing message producer in organization service 286–291

synchronous request-response approach to communicate state change 277–279

brittleness between services 278

inflexibility in adding new consumers to changes in organization service 278–279

tight coupling between services 278

efficient storage 395

EKS (Elastic Kubernetes Service) 345

Elastic Container Registry (ECR) 358, 365

Elastic Container Service (ECS) 16

elasticity 18

ELK (Elasticsearch, Logstash, and Kibana) stack

configuring Kibana 320–323

configuring logback in services

adding Logstash encoder 314–315

creating Logstash TCP Appender 315–318

creating EC2 instances 353–356

creating EKS clusters 357–364

deploying microservices in EKS cluster 361–364

provisioning Kubernetes cluster 358

pushing microservices images into repositories 358–360

defining and running ELK stack applications in Docker 318–320

deploying in EC2 instance 356–357

implementation 313–314

searching for trace IDs in Kibana 323

@EnableBinding annotation 287, 294

EnableBinding annotation 294

@EnableDiscoveryClient annotation 171

@EnableEurekaClient annotation 36, 215

@EnableEurekaServer annotation 162

@EnableFeignClients annotation 174

/encrypt endpoint 146

ENCRYPT_KEY variable 145

encryption

encrypting and decrypting property 146–147

@Entity annotation 134

ENTRYPOINT command 102

environment section 329

ES_HOSTS variable 329

Eureka 33

building Spring Eureka service 158–162

configuring Spring Cloud Gateway to communicate with 215–216

invoking services with Netflix Feign client 174–175

registering services with Spring Eureka 162–167

REST API 166

service discovery 156–157

Eureka server 158

eureka.client.fetchRegistry attribute 161, 165

eureka.client.registerWithEureka attribute 161, 165

eureka.client.serviceUrl.defaultZone attribute 161

eureka.instance.hostname attribute 161

eureka.server.waitTimeInMsWhenSyncEmpty attribute 161

eureka.serviceUrl.defaultZone attribute 165

EurekaServerApplication class 161

event-driven architecture. See EDA (event-driven architecture)

expires_in attribute 258

externalized configuration 380

F

FaaS (Function as a Service) 16

failureRateThreshold 193

fallback processing pattern

implementing 193–194

overview 180

fallbackMethod attribute 194

fault tolerant service discovery 152

Filter class 202

Filter-Utils 271

filter() method 229–230, 271

filters

building post-filter receiving correlation IDs 237–238

building pre-filters 228–231

custom 225–228

Filters Factories 222–224

UserContext 234–235

UserContextFilter 233–234

Filters Factories

built-in 224

overview 222–223

FilterUtils class 230

FilterUtils setCorrelationId() method 230–231

findById() method 337

firstName parameter 13

flexible query language 395

FROM command 102

Function as a Service (FaaS) 16

G

getCorrelationId() method 230

getInstances() method 172

getLicense() method 69, 79–80, 137, 170

getLicenses() method 205

getLicensesByOrganization() method 189–190, 197, 205

getOrganization() method 175, 303

getRestTemplate() method 173

getUsername() method 271

Git 33, 139–140

GitHub 365–367

GKE (Google Container Engine) 16

Global-Filter class 229

Globally Universal ID (GUID) 228

Google Container Engine (GKE) 16

Grafana, configuring 398–402

grant_type parameter 256

grants 241

granularity, establishing 59–61

group property 295

GUID (Globally Universal ID) 228

H

HATEOAS (Hypertext as the Engine of Application State) 379

highly available service discovery 152

host type 95

HTTP

adding correlation ID to response with Spring Cloud Gateway 325–326

intercepting incoming request 233–234

making headers easily accessible to service 234–235

using HTTPS SSL for all service communication 273

HTTPRequest 71

HttpSecurity class 262

Hypertext as the Engine of Application State (HATEOAS) 379

I

IaaS (Infrastructure as a Service) 16

@Id annotation 134

IDE (Integrated Development Environment) 117

IdP (identity provider) 241–242

ignoreExceptions parameter 199

implicit grant types 388–390

inboundOrgChanges channel 305

Infrastructure as a Service (IaaS) 16

@Input annotation 304

input section 319

Integrated Development Environment (IDE) 117

internationalization, applying into licensing service 74–77

intervalFunction parameter 199

inverse Conway maneuver 7

J

JAR_FILE variable 102

jarmode system property 106

JARs

adding Spring Security and Keycloak JARs to individual service 259

Java Persistence API (JPA) 129

Java, building microservices with 47–53, 63–80

applying internationalization into licensing service 74–77

booting application 52–53

building doorway into microservice 64–74

implementing Spring HATEOAS to display related links 78–80

setting up environment 47

skeleton project 47–51

Jenkins 365, 367–372

JPA (Java Persistence API) 129

JSON (JavaScript Object Notation) 5

JWTs (JSON Web Tokens) 25, 35, 242, 270–272

K

KeepAliveDuration 197

Keycloak

overview 242–244

protecting organization service 258–272

adding Spring Security and Keycloak JARs to individual service 259

configuring service to point to Keycloak server 260

defining who and what can access service 260–265

parsing custom field in JWT 270–272

propagating access token 265–270

protecting single endpoint 244–258

adding Keycloak to Docker 245

authenticating O-stock users 255–258

configuring O-stock users 253–255

registering client application 248–252

setting up Keycloak 246–247

KeycloakRestTemplate class 268–269

KeycloakWebSecurityConfigurerAdapter class 260

Kibana

configuring 320–323

searching for trace IDs in 323

Kibana Query Language (KQL) 323

kompose up command 362

KQL (Kibana Query Language) 323

kubeconfig (Kubernetes Configuration) 369

kubectl get nodes -o wide command 363

kubectl get pods command 362

kubectl get svc command 358

Kubernetes

creating pipeline scripts 374

provisioning cluster 358

L

lastName parameter 13

layertools 106

least privilege 273

license .withComment() value 137

License class 68, 70, 79–80, 133–134

License object 70, 194

License-Repository interface 136

LicenseController class 64, 79, 169, 206

LicenseController.java class 205

licenseId parameter 70

LicenseRepository class 135

LicenseService class 137, 170, 206

LicenseService-Application class 53

LicenseServiceApplication class 173, 236, 294

LicenseServiceApplication.java class 75

licensing-service 164

licensingGroup 295

licensingService class 136

limitForPeriod parameter 201

linkTo method 80

load balanced service discovery 152

load balancing pattern 179–180

@LoadBalanced annotation 173

LoadBalancer service 362

Locale 75, 77

LocaleResolver 75

log aggregation 311

adding correlation ID to HTTP response with Spring Cloud Gateway 325–326

adding Logstash encoder 314–315

configuring Kibana 320–323

creating Logstash TCP Appender 315–318

defining and running ELK stack applications in Docker 318–320

searching for trace IDs in Kibana 323

Spring Cloud Sleuth/ELK stack implementation 313–314

log correlation 26

loggerSink() method 294

logging 45, 382

logging patterns 26

Logging-EventCompositeJsonEncoder 317

LoggingEventCompositeJsonEncoder class 315

Logstash

adding encoder 314–315

creating Logstash TCP Appender 315–318

logstash-logback-encoder dependency 315

LogstashEncoder class 315–316

M

macvlan type 95

main() method 53, 120

manningPublications tag 319

Mapped Diagnostic Context (MDC) 309

maven command 373

Maven plugin 365

maxConcurrentCalls property 197

maxThreadPoolSize property 197

maxWaitDuration property 197

MDA (message-driven architecture) 276

MDC (Mapped Diagnostic Context) 309

Message class 290

MessageBuilder class 290

MessageChannel class 289–290, 304

messageSource 75

messaging 279–281

downsides of 281–282

choreography 282

handling semantics 281

visibility 281

durability 280

flexibility 281

loose coupling 280

scalability 280–281

method parameter 69

metrics 27

metrics service 27

metrics visualization suite 27

Micrometer

implementing 395–397

setting up 394–397

microservices 30, 90

architect's role 56–62

decomposing business problem 56–59

defining service interfaces 61–62

establishing service granularity 59–61

architecture best practices 377–382

API Gateways 382

CI/CD 380–381

externalized configuration 380

logging 382

monitoring 381

Richardson Maturity Model 378–380

Spring HATEOAS 380

building with Spring Boot and Java 47–53

booting application 52–53

setting up environment 47

skeleton project 47–51

client communication with 85–86

cloud computing 11–19

building microservices with Spring Boot 11–14

defined 16–17

reasons for deploying microservices to cloud 18–19

defined 4–5

deploying 376

build/deployment pipeline 376

setting up core infrastructure in cloud 346–352

DevOps engineer's role 81–89

communicating microservice health 86–89

service assembly 83–84

service bootstrapping 84–85

service registration and discovery 85–86

evolution towards microservices architecture 2–7

guidelines for 19–21

integrating Docker with 99–108

creating Docker Images 99–104

launching services with Docker Compose 107–108

managing configuration 84–85

monitoring 392–402

Grafana 398–402

Micrometer and Prometheus 394–397

Spring Boot Actuator 393

monolithic architecture 3–4

n-tier architecture 2

overview of book 10–11

relevance of material 10–11

relevant examples 46

scope of material 10

packaging and deploying 83–84

patterns

application metrics pattern 27–28

build and deployment patterns 28

client resiliency patterns 23–24

core development pattern 21–22

logging and tracing patterns 26

routing patterns 22–23

security patterns 24–25

pushing images into repositories 358–360

reasons for switching to 6–7

security 274

Keycloak 242–272

locking down unneeded network ports 274

OAuth2 241–242

using services gateway to access 273

zoning services into public API and private API 273

software engineer's role 63–80

applying internationalization into licensing service 74–77

building doorway into microservice 64–74

implementing Spring HATEOAS to display related links 78–80

twelve-factor app best practices 37–46

admin processes 45–46

backing services 42

build, release, run 42–43

codebase 40

concurrency 44

config 41

dependencies 40

dev/prod parity 44–45

disposability 44

logs 45

port binding 43

processes 43

when not to use 62–63

application type 63

complexity when building distributed systems 62

data transactions and consistency 63

server or container sprawl 63

with Spring 7–8

monitoring microservices 381, 392–402

Grafana 398–402

Micrometer and Prometheus

implementing 395–397

setting up 394–397

Spring Boot Actuator

adding 393

enabling endpoints 393

monolithic architecture 3–4

multidimensional data models 395

multiple integrations 395

mvn command 320

mvn spring-boot:run command 13, 124, 162

N

n-tier architecture 2

NanosToWait setting 200

net.logstash.logback.encoder.LoggingEvent-CompositeJsonEncoder class 315

net.logstash.logback.encoder.LogstashEncoder class 315

Netflix Eureka. *See* Eureka

Netflix Feign client, invoking services with 174–175

nodePort attribute 362

NodePort service 362

none type 95

O

O-stock

authenticating users 255–258

configuring users 253–255

deploying 353–364

creating an EC2 with ELK 353–356

creating EKS clusters 357–364

deploying ELK stack in EC2 instance 356–357

setting up core infrastructure in cloud 346–352

creating PostgreSQL database using Amazon RDS 348–350

creating Redis cluster in Amazon 351–352

OAuth2

grant types 383–390

authorization grant types 386–388

client credential grant types 385

implicit grant types 388–390

password grant types 384–385

refreshing tokens 390

overview 241–242

OIDC (OpenID Connect) 241–242

/openid-connect/token endpoint 262

Organization class 175

Organization object 303

organization-service property 164

OrganizationChangeHandler class 306

OrganizationChangeModel class 289–290, 294

OrganizationFeignClient class 175

organizationId element 290

organizationId parameter 70, 175

OrganizationRestTemplateClient class 269, 302

OrganizationService class 291, 337

OrganizationServiceApplication class 287

orgChangeTopic 334

output() method 289

@OutputChannel annotation 304

overlay type 95

P

PaaS (Platform as a Service) 16

password grant types 384–385

password parameter 256

path variables 11

@PathVariable annotation 13, 69, 71, 175

patterns

application metrics pattern 27–28

build/deployment patterns 28

client resiliency patterns 23–24

core development pattern 21–22

logging and tracing patterns 26

routing patterns 22–23

security patterns 24–25

peer-to-peer service discovery 152

PEP (Policy Enforcement Point) 209, 273

physical servers 18

Platform as a Service (PaaS) 16

POJO (Plain Old Java Object) 127, 284

Policy Enforcement Point (PEP) 209, 273

port binding 43

portability 91

post-filters 226

PostgreSQL database, creating using Amazon RDS 348–350

postgresql dependency 129

@PostMapping annotation 71

pre-filters 225

Predicates Factories

built-in 223

overview 222–223

processes 43

profiles 130

Prometheus

implementing 395–397

setting up 394–397

property attribute 138

protected resource 243

publishOrganizationChange() method 289

@PutMapping annotation 71

Q

queueCapacity property 197

R

rate limiter pattern, implementing 200–202

RateLimiter 200

@RateLimiter annotation 205

readLicensingDataFromRedis 336

realms 246

recordExceptions 193

Redis

configuring in Docker 285

creating cluster in Amazon 351–352

using to cache lookups 298–304

configuring licensing service 298

constructing database connection to Redis server 298–300

defining Spring data Redis repositories 300–301

using Redis and licensing service to store and read organization data 302–304

Redis-Template object 299

/refresh endpoint 138

refresh_token attribute 258

@RefreshScope annotation 138

remote procedure call (RPC) 378

RemoveRequestHeader 266

@Repository annotation 53, 135

RepresentationModel 80

@RequestBody annotation 71

@RequestHeader annotation 77

@RequestMapping annotation 66, 69, 175

Resilience4j

implementing 185

setting up licensing service to use 186–187

ThreadLocal and 202–207

resilience4j-circuitbreaker artifact 186

resilience4j-spring-boot2 artifact 186

resilience4j-time-limiter artifact 186

resiliency patterns 207

bulkhead pattern 180–181, 195–198

circuit breaker pattern 180, 187–193

adding to organization service 191–192

customizing 192–193

fallback processing pattern 180, 193–194

importance of 181–184

load balancing pattern 179–180

rate limiter pattern 200–202

Resilience4j

implementing 185

setting up licensing service to use 186–187

ThreadLocal and 202–207

retry pattern 198–200

resilient service discovery 152

resource owner 243

ResourceBundleMessageSource 75

response filter 228

@ResponseBody annotation 65

ResponseBody class 65

ResponseEntity object 70

ResponseFilter 238, 326

REST (Representational State Transfer) 7

REST endpoint 126

Rest-Template class 170

@RestController annotation 65

restTemplate.exchange() 174

retrieveOrganizationInfo() method 170

@Retry annotation 205

retry pattern 198–200

retryOnExceptionPredicate parameter 199

retryOnResultPredicate parameter 199

Richardson Maturity Model 378–380

ringBufferSizeInClosedState 193

ringBufferSizeInHalfOpenState 193

@RolesAllowed annotation 263

root token 141

route configuration 216–222

automated, via service discovery 216–218

dynamically reloading 221–222

manual, using service discovery 218–221

/routes endpoint 221

routing patterns 22–23

RPC (remote procedure call) 378

S

SaaS (Software as a Service) 17

SAML (Security Assertion Markup Language) 241

Sampler class 330

SCA (Software Composition Analysis) 241

scope attribute 258

search-locations parameter 122

security 274

Keycloak

overview 242–244

protecting organization service 258–272

protecting single endpoint 244–258

locking down unneeded network ports 274

OAuth2 241–242

using HTTPS SSL for all service communication 273

using services gateway to access microservices 273

zoning services into public API and private API 273

Security Assertion Markup Language (SAML) 241

security patterns 24–25

SecurityConfig class 260, 267

SecurityConfig.java class 261

segregate principle 112

semaphore.release 200

semaphore.tryAcquire method 200

SemaphoreBasedRateLimiter 200

send() method 290

server sprawl 63

server.port attribute 161

ServerWebExchange object 229

ServerWebExchange.Builder mutate() method 231

ServerWebExchangeDecorator 231

@Service annotation 53

service assembly 83–84

service bootstrapping 84–85

service discovery 85–176

building Spring Eureka service 158–162

in cloud 152–157

architecture 153–156

using Spring and Netflix Eureka 156–157

looking up services 169–175

invoking services with Load Balancer aware Spring REST template 173–174

invoking services with Netflix Feign client 174–175

looking up service instances with Spring Discovery Client 171–172

overview 149–152

registering services with Spring Eureka 162–167

REST API 166

service interfaces, defining 61–62

service registration and discovery 85–86

service routing 239

building post-filter receiving correlation IDs 237–238

building pre-filters 228–231

custom filters 225–228

Filters Factories

built-in 224

overview 222–223

Predicates Factories

built-in 223

overview 222–223

route configuration 216–222

automated 216–218

dynamically reloading 221–222

manual 218–221

Spring Cloud Gateway

configuring to communicate with Eureka 215–216

defined 209–211

setting up project 212–214

UserContext 234–235

UserContextFilter 233–234

Service-Instance class 172

ServiceConfig class 138, 299

ServiceInstance class 172

ServiceInstance objects 172

ServletFilter 232

Set-Cookie value 266

set-CorrelationId() method 230

SimpleSourceBean class 290

Sink interface 294

Sink.class 294

Sink.INPUT channel 295

Software as a Service (SaaS) 17

Software Composition Analysis (SCA) 241

software engineer's (developer's) role 63–80

applying internationalization into licensing service 74–77

building doorway into microservice 64–74

implementing Spring HATEOAS to display related links 78–80

Source class 287, 289

Source interface 294

source.output() method 290

spanExportable field 316

SpanId field 316

Spotify Docker plugin 365

spring .application.name property 160, 164

Spring Boot

building microservices with 11–14, 47–53, 63–80

applying internationalization into licensing service 74–77

booting application 52–53

building doorway into microservice 64–74

implementing Spring HATEOAS to display related links 78–80

setting up environment 47

skeleton project 47–51

Buildpacks 105

Spring Boot Actuator

adding 393

enabling endpoints 393

Spring Boot client, integrating Spring Cloud Configuration Server with 127–144

configuring licensing service to use Spring Cloud Config 129–132

directly reading properties 137–138

integrating Vault with 141

refreshing properties 138–139

setting up licensing service dependencies 128–129

using with Git 139–140

Vault UI 141–144

wiring in data source 133–137

Spring Boot controller 64–74

Spring Cloud

defined 32–33

Hello World example 35–37

service discovery 156–157

building Spring Eureka Service 158–162

registering services with Spring Eureka 162–167

setting up licensing service to use 186–187

tools

Spring Cloud API Gateway 34

Spring Cloud Config 33

Spring Cloud LoadBalancer and Resilience4j 33–34

Spring Cloud Security 35

Spring Cloud Service Discovery 33

Spring Cloud Sleuth 34

Spring Cloud Stream 34

Spring Cloud API Gateway 34

Spring Cloud Configuration Server 33–147

building 115–126

setting up bootstrap class 120

setting up configuration files for service 122–126

using with filesystem 121–122

configuration management architecture 112–114

implementation choices 114–115

integrating with Spring Boot client 127–144

configuring licensing service to use Spring Cloud Config 129–132

directly reading properties 137–138

integrating Vault with 141

refreshing properties 138–139

setting up licensing service dependencies 128–129

using with Git 139–140

Vault UI 141–144

wiring in data source 133–137

protecting sensitive configuration information 144–147

encrypting and decrypting property 146–147

Spring Cloud Gateway 239

adding correlation ID to HTTP response with 325–326

building post-filter receiving correlation IDs 237–238

building pre-filters 228–231

configuring to communicate with Eureka 215–216

custom filters 225–228

defined 209–211

Filters Factories

built-in 224

overview 222–223

Predicates Factories

built-in 223

overview 222–223

route configuration 216–222

automated 216–218

dynamically reloading 221–222

manual 218–221

setting up project 212–214

UserContext 234–235

UserContextFilter 233–234

Spring Cloud LoadBalancer 33–34, 173–174

Spring Cloud Security 35

Spring Cloud Service Discovery 33

Spring Cloud Sleuth 34

correlation ID and 309–311

adding Spring Cloud Sleuth to licensing and organization 309–310

anatomy of trace 310–311

log aggregation and 311–326

adding correlation ID to HTTP response with Spring Cloud Gateway 325–326

adding Logstash encoder 314–315

configuring Kibana 320–323

creating Logstash TCP Appender 315–318

defining and running ELK stack applications in Docker 318–320

searching for trace IDs in Kibana 323

Spring Cloud Sleuth/ELK stack implementation 313–314

setting up dependencies 328

Spring Cloud Stream 34, 282–306

configuring Apache Kafka and Redis in Docker 285

defining custom channels 304–306

using Redis to cache lookups 298–304

configuring licensing service 298

constructing database connection to Redis server 298–300

defining Spring data Redis repositories 300–301

using Redis and licensing service to store and read organization data 302–304

writing message consumer in licensing service 293–295

writing message producer in organization service 286–291

Spring Discovery Client, looking up service instances with 171–172

Spring Eureka

building service 158–162

registering services with 162–167

REST API 166

Spring HATEOAS (Hypermedia as the Engine of Application State) 78–80, 380

Spring REST template, invoking services with 173–174

SPRING_PROFILES_ACTIVE variable 131

SPRING_CLOUD_CONFIG_URI variable 131

spring-boot-starter-aop dependency 186

spring-boot-starter-data-jpa dependency 129

spring-cloud-sleuth-zipkin dependency 328

spring-cloud-starter-config dependency 129

spring-cloud-starter-netflix-eureka-client dependency 36

spring-cloud-starter-sleuth dependency 325

spring-data-redis dependency 298

Spring, microservices with 7–8

spring.application.name property 130, 163, 215

spring.cloud. config.server.git property 140

spring.cloud.config.server.git.searchPaths property 140

spring.cloud.config.server.git.uri property 140

spring.cloud.config.uri property 130

spring.datasource.password property 146

spring.profiles.active property 130, 140

spring.sleuth.sampler.percentage property 330

spring.zipkin.baseUrl property 328

@SpringBootApplication annotation 53

SSO (single sign-on) 242, 248

STORAGE_TYPE variable 329

@StreamListener annotation 294

SubscribableChannel class 304

symmetric encryption key

encrypting and decrypting property 146–147

synchronous request-response approach 277–279

brittleness between services 278

inflexibility in adding new consumers to changes in organization service 278–279

tight coupling between services 278

T

@Table annotation 134

target service 228

Thread-Local variable 228

ThreadLocal 202–207, 288

tmx-correlation-ID header 230

tmx-correlation-id header 205, 228, 238, 326, 334

token_type attribute 257

TraceId field 316

Tracer class 326

tracer.currentSpan().context() .traceIdString() method 326

tracing patterns 26

tracking filter 228

TrackingFilter 228, 230, 232

twelve-factor app 39

type–NodePort attribute 362

U

UDDI (Universal Description, Discovery, and Integration) 148

unbundling 4

updateLicense() method 71

UserContextHolder class 203

UserContext class 204, 206, 232–235, 288

UserContext object 202–203

UserContextFilter class 232–233

UserContextHolder class 203–204, 235

V

value attribute 66

value parameter 69

Vault

integrating Spring Cloud Configuration Server with 141

UI 141–144

VAULT_DEV_LISTEN_ADDRESS parameter 141

VAULT_DEV_ROOT_TOKEN_ID parameter 141

velocity 341

version control 40

virtual container 18

virtual machine images 18

VMs (virtual machines) 18, 38, 92–94

VPC (Virtual Private Cloud) 349

W

waitDurationInOpenState 193

X

X-B3-SpanId field 316

X-B3-TraceId field 316

Z

Zipkin 327–338

adding custom spans 336–338

capturing messaging traces 334–335

configuring server 328–329

configuring services to point to 328

setting tracing levels 329–330

setting up dependencies 328

tracing transactions 330–333

visualizing more complex transactions 333

Previous chapter
‹ appendix C. Monitoring your microservices

Next chapter
inside back cover ›